Quantifying the Computational Cost of STDP in Spiking Neural Networks: A Benchmark Study Towards FPGA Acceleration

Maxime Carriere — Brain Language Laboratory, Freie Universität Berlin

March 2026 — Preliminary results

Abstract

Spiking Neural Networks (SNNs) trained with Spike-Timing Dependent Plasticity (STDP) offer a biologically plausible model of learning. Their simulation, however, carries a heavy computational cost that grows super-linearly with network size. Using the NEST simulator across 2,160 configurations (network size, connection density, firing rate, and core count), we show that STDP can more than **double** total simulation time compared to static synapses—an overhead reaching **183%** for large networks. Multi-core CPU parallelism scales both synaptic rules equally but cannot close the STDP overhead gap: STDP remains ~2x slower than static at all core counts. These findings motivate a hardware-accelerated solution. We present these benchmarks as a foundation for designing an FPGA-based STDP co-processor.

1. Introduction

The human brain processes sensory information using approximately 10^{11} neurons connected by roughly 10^{15} synapses [1]. Unlike the artificial neural networks behind today's deep-learning revolution—which use smooth continuous activations and gradient-based training—biological neurons communicate through discrete electrical pulses called *spikes*. Models that respect this mechanism are called **Spiking Neural Networks (SNNs)**.

SNNs are of great interest for neuromorphic computing, low-power embedded intelligence, and fundamental neuroscience research. However, simulating even small-scale networks on conventional computers is expensive. The key bottleneck is the learning rule that governs how synaptic connections are updated during simulation.

In this work we focus on the dominant biological learning rule: **Spike-Timing Dependent Plasticity (STDP)**. We systematically benchmark its computational overhead using the widely used NEST simulator [2] and identify the scaling regimes where conventional CPUs become inadequate. Our ultimate goal is to offload STDP weight updates to an FPGA co-processor, enabling real-time simulation of large networks.

2. Background: What is STDP?

2.1 Synaptic Plasticity

Learning in biological neural circuits is implemented by changing the *strength* (weight) of synaptic connections. A synapse connects a *pre-synaptic* neuron (the sender) to a *post-synaptic* neuron (the receiver). The key insight from Hebbian theory is: *neurons that fire together wire together* [3].

2.2 The STDP Rule

STDP refines this idea by making the weight change depend on the *relative timing* of pre- and post-synaptic spikes, first demonstrated experimentally by Markram et al. [4] and subsequently characterised in hippocampal neurons by Bi and Poo [5]. Define $dt = t_{post} - t_{pre}$. The weight update follows an exponential learning window (Fig. 1):

dW = A+ x exp(-dt/tau+) if dt >= 0 [LTP]

dW = -A- x exp(+dt/tau-) if dt < 0 [LTD]

When the pre-synaptic neuron fires *before* the post-synaptic one (dt > 0), the synapse is **strengthened** (Long-Term Potentiation, LTP). In the opposite case it is **weakened** (Long-Term Depression, LTD). Time constants tau± approx. 20 ms govern the time window over which coincidences are detected.

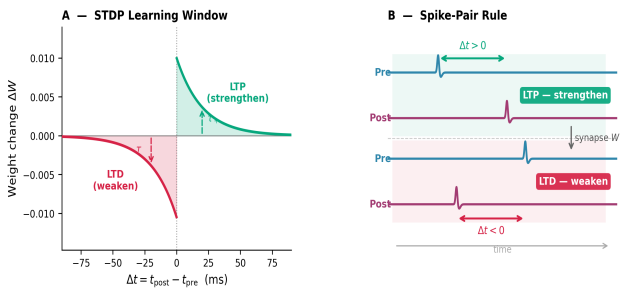

Figure 1. STDP learning window. (A) Weight change dW vs inter-spike interval dt. Pre fires before post (dt>0): LTP (green). Post fires before pre (dt<0): LTD (red). (B) Spike-pair schematic showing the synapse W between two neurons.

2.3 Computational Cost

For a network of N neurons, every spike triggers a weight update at each connected synapse. The total number of STDP operations per second scales as:

Updates/s = f · N · k_in

where f is the mean firing rate and k_in is the mean in-degree (average number of incoming synapses per neuron). With N = 2,000, f = 20 Hz, and connection probability 10%, this yields approx. 8×10^6 updates/second — all serialised on a CPU, incurring memory-bandwidth pressure that grows super-linearly with N, as shown below.

3. Experimental Setup

We used **NEST 3.6** [2] to simulate recurrent networks of integrate-and-fire neurons (iaf_psc_alpha) driven by Poisson external input. Each configuration was simulated for 20 s of biological time. The parameter space covered:

- Network size N: 10 to 16,000 neurons (10 values, log-spaced)
- CPU cores: 1, 2, 4, 8
- Connection probability p: 5%, 10%, 20%

- External firing rate: 10, 20, 40 Hz
- Learning rule: static synapses vs additive STDP (tau± = 20 ms, lambda = 0.01)

In total, **2,160 simulations** were run (3 trials each for statistical error bars), collected over 2.9 days on a standard desktop workstation. Timing was measured with time.perf_counter(), separating the network setup phase from the simulation phase.

4. Results

4.1 Overhead is Negligible Below 1,000 Neurons, Then Critical

Fig. 2 compares simulation time for static and STDP networks (1 core, p = 0.1, 20 Hz). For small networks (N <= 500) both rules complete in under 5 s and bars are nearly identical (Fig. 2A). The linear-scale plot (Fig. 2B) reveals a sharp transition around N ~ 1,000 neurons, above which the two curves diverge dramatically: **1.9× at N = 2,000** (53 s vs. 28 s), **2.3× at N = 4,000** (177 s vs. 76 s), and **2.4× at N = 8,000** (808 s vs. 330 s) — an absolute overhead of nearly **8 minutes per simulation**.

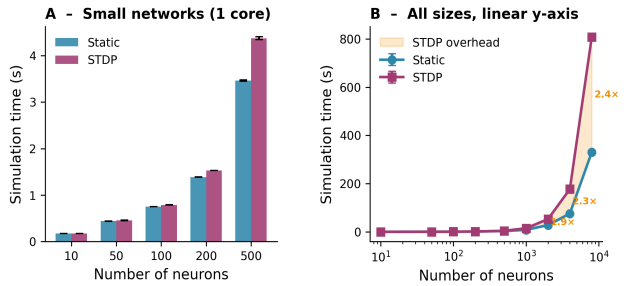

Figure 2. Static vs STDP simulation time. (A) Bar chart for small networks (N <= 500). (B) Full log-scale comparison; shaded region marks STDP overhead. 1 core, p = 0.1, 20 Hz Poisson input. Points = mean of 3 trials; error bars = SEM.

4.2 Why Overhead Peaks, Then Drops — and What the Per-Synapse Cost Reveals

Fig. 3A — Overhead percentage. For all connection densities, STDP overhead is negligible (below 5%) for very small networks (N < 100) and grows steeply beyond N ~ 500, peaking at **183%** for dense networks around N = 4,000. A 183% overhead means STDP takes **2.83× as long** as the static baseline (total = static + 183% of static).

The **drop for p = 0.20 at N = 8,000** (red curve, Fig. 3A) is not a measurement error — it is a real network effect. At that size and density, neurons excite each other in a runaway chain reaction, driving the firing rate to 162 Hz — eight times the 20 Hz input signal. Think of it as a feedback loop: more spikes trigger more spikes. The *static* simulation must track all of these extra spikes (its runtime jumps 7.8× from N = 4,000 to 8,000), while STDP weight updates only grow 1.9× over the same step because the weight arithmetic is bounded. The overhead *percentage* therefore shrinks — not because STDP became cheaper (it still adds 434 s of absolute extra cost), but because the baseline exploded even faster. In biologically realistic models, networks are kept below such runaway regimes, placing them in the 100–170% overhead zone.

Fig. 3B — Cost per synapse. If every synapse update took the same time regardless of network size, the curve would be flat (dashed reference). Instead, three regimes appear (Fig. 3B), explained by a fundamental property of modern processors:

A CPU does not read data directly from RAM each time it needs it — that would be too slow. Instead, it keeps a small **cache** of recently used data in fast memory built right next to the processor chip. Think of it like a desk: the cache is the small desk surface where you keep the papers you are actively using, and RAM is the filing cabinet across the room. Working from the desk is fast; getting up to fetch a file from the cabinet takes much longer. A **cache miss** is exactly that: the processor needs a piece of data, finds it is not on the desk, and must walk to the cabinet — wasting time waiting instead of computing.

- **Zone 1 — Very small networks (N < 100, blue):** Cost is ~60 µs per synapse, even though the table is tiny. The reason is fixed overhead per update event: every STDP calculation involves a function call, a timestamp lookup, and housekeeping that costs roughly the same regardless of how many synapses exist. With only ~10 synapses, this overhead is never amortised.
- **Zone 2 — Medium networks (N = 100–800, green):** Cost drops to ~36 µs — the cheapest regime. The synapse table is small enough (under ~1 MB) to fit entirely in the processor fast cache (the desk). Every weight lookup is served instantly from cache, no waiting for RAM.
- **Zone 3 — Large networks (N >= 800, red):** Cost doubles to ~63 µs. The synapse table now exceeds the cache size (100k synapses × ~32 bytes = ~3 MB). The processor must fetch most weights from RAM — a cache miss on almost every update. It spends more time waiting for data than actually computing, which is why the cost per synapse is nearly twice as high as in Zone 2.

This boundary at N ~ 800 explains the sharp rise in overhead seen in Fig. 3A. An FPGA eliminates this problem entirely: it stores the synapse table in dedicated on-chip memory (called Block RAM) that is as fast as a cache regardless of table size — there is no filing cabinet to walk to. The ~2x cost penalty of Zone 3 would vanish.

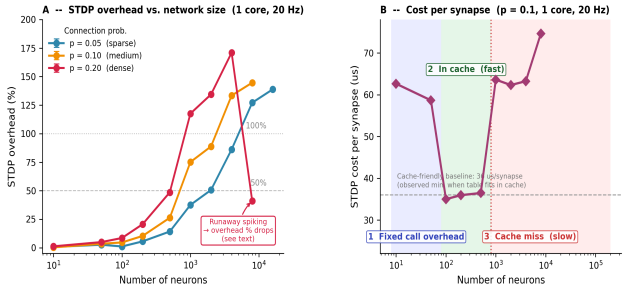

Figure 3. (A) STDP overhead (%) vs. network size for three connection densities (1 core, 20 Hz). The annotated drop for p = 0.20 at N = 8,000 reflects network runaway excitation, not a reduction in STDP cost. (B) STDP cost per synapse. Three regimes are highlighted: fixed per-event overhead (blue), in-cache access (green, fast zone), and cache-miss zone (red, slow zone). The dashed line is the ideal constant cost.

4.3 Multi-Core Parallelism Does Not Close the STDP Gap

A natural first response to the STDP overhead is to simply use more CPU cores: if STDP doubles the computation, perhaps running on two cores restores the original speed. Fig. 4 tests this intuition directly.

Speedup with more cores (Panel A). For static synapses, parallelism is beneficial only for large networks. Below N = 500 neurons, adding cores barely helps or even hurts: coordinating multiple threads requires constant communication, and for small workloads this cost exceeds the time saved by splitting the work. At 10 neurons, 8 cores are actually 1.8x *slower* than a single core. For large networks (N >= 2,000), multi-core scaling is substantial, reaching close to or above ideal linear scaling, with a practical ceiling around 10x speedup at 8 cores.

The STDP overhead gap persists at every core count (Panel B). Panel B shows the raw wall-clock simulation time for both static and STDP synapses at N = 4,000 neurons -- the most informative way to compare the two learning rules under parallelism. Both curves decrease steeply as cores are added, confirming that parallelism is effective for both rule types. Going from 1 to 8 cores reduces static time from 76 s to 7.8 s (9.7x faster) and STDP time from 177 s to 16 s (11.0x faster).

Yet at every single core count, STDP remains approximately 2.1-2.3x slower than static, as shown by the bracket at 8 cores. The two curves run in parallel: they drop at the same rate, keeping the same proportional distance. **Adding more CPU cores cannot eliminate the STDP overhead.** Each core still has to execute the same weight-update calculation for every synapse; parallelism simply divides this work among more processors, but does not make the calculation itself any cheaper.

Think of it this way: if STDP requires ten steps where static requires four, then splitting across eight cores gives you eight workers each doing ten steps vs. four steps. Every worker is still 2.5x slower on their share of the work. The ratio never changes.

This finding has an important practical implication. Scaling from 1 to 8 cores on a typical server is already a significant hardware investment. Yet even with this investment, a biologically realistic network with STDP synapses will always simulate 2x slower than the same network with fixed weights. For real-time or near-real-time brain-computer interface applications, this gap is unacceptable. A fundamentally different solution is needed -- one that reduces the cost of the STDP computation itself rather than distributing it more efficiently across more processors.

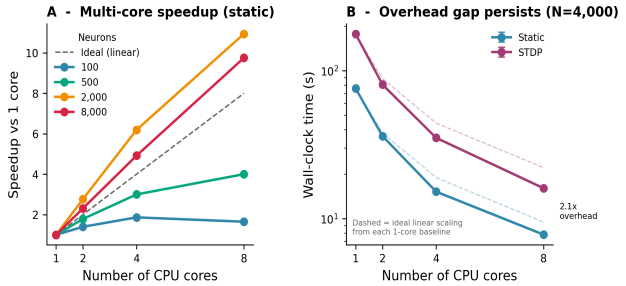

Figure 4. Multi-core scaling. (A) Speedup relative to 1 core for different network sizes (static synapses, p = 0.1, 20 Hz); dashed line = ideal linear scaling. Small networks gain little or nothing from additional cores. (B) Absolute wall-clock simulation time for static and STDP at N = 4,000. Both rules scale similarly with core count (faint dashed = ideal from each 1-core baseline), but the STDP overhead of ~2.1x persists regardless of core count. More cores do not close the gap.

5. The Case for FPGA Acceleration

The results above identify a clear bottleneck: STDP weight updates incur a super-linearly growing cost that neither multi-core CPUs nor software optimisation can fully address. The root cause is architectural: a general-purpose CPU must (1) *fetch* the synapse record from DRAM (cache-miss prone), (2) *compute* the exponential decay, and (3) *write back* the updated weight — millions of times per second, on a single sequential pipeline.

FPGAs (Field-Programmable Gate Arrays) offer a compelling alternative:

- **Massive parallelism**: process hundreds of synaptic updates simultaneously in dedicated hardware lanes.
- **On-chip BRAM**: synapse tables for moderate networks fit in fast on-chip memory, eliminating DRAM latency.
- **Fixed-point exponentials**: the STDP kernel maps efficiently to lookup tables (LUTs).
- **Energy efficiency**: FPGA neural simulation consumes 10–100× less power than GPU or CPU equivalents [5,6].
- **Deterministic latency**: cycle-accurate pipelines simplify spike-timestamp bookkeeping required by STDP.

Several FPGA-based SNN accelerators have been proposed [5,6,7], but none directly targets the STDP update pipeline as a co-processor to a CPU-based simulator such as NEST. Our planned architecture places the FPGA alongside the host CPU: the CPU manages network topology and spike routing, while the FPGA handles all weight-update arithmetic. The benchmarks presented here

will guide the memory-bandwidth and arithmetic-throughput requirements for that co-processor design.

6. Conclusion

We have characterised the computational overhead of STDP in the NEST simulator across a wide range of network sizes, densities, and hardware configurations. Key findings:

- **Below ~1,000 neurons**: overhead is negligible (<10%); standard CPUs are sufficient.
- **Above ~1,000 neurons**: sharp transition — overhead exceeds 50% at 500 neurons and peaks at **183%** (2.83× as long) around 4,000 neurons.
- **Absolute cost**: ~8 minutes of extra runtime per simulation at 8,000 neurons — prohibitive for sweeps.
- **CPU parallelism limited**: synchronisation and irregular memory access prevent effective scaling.
- **Root bottleneck is memory access**: the synapse table outgrows the CPU cache, causing costly RAM fetches. FPGA on-chip memory eliminates this penalty.

These results establish a quantitative baseline for the design and evaluation of an FPGA-based STDP accelerator, which is the next step of this work.

Data and Code Availability

All simulation scripts, processed data (CSV), and figure-generation code are publicly available at:

https://github.com/MaximeCarriere/nest-stdp-benchmark

The repository includes step-by-step instructions to reproduce every figure in this paper from scratch using the NEST simulator, or to regenerate the figures directly from the pre-computed CSV data without running new simulations.

References

[1] F. A. C. Azevedo et al., "Equal numbers of neuronal and nonneuronal cells make the human brain an isometrically scaled-up primate brain," J. Comp. Neurol., 2009.

[2] M.-O. Gewaltig and M. Diesmann, "NEST (NEural Simulation Tool)," Scholarpedia, vol. 2, no. 4, p. 1430, 2007.

[3] D. O. Hebb, The Organization of Behavior. Wiley, 1949.

[4] H. Markram, J. Lübke, M. Frotscher, B. Sakmann, "Regulation of synaptic efficacy by coincidence of postsynaptic APs and EPSPs," Science, vol. 275, pp. 213–215, 1997.

[5] G.-Q. Bi and M.-M. Poo, "Synaptic modifications in cultured hippocampal neurons," J. Neurosci., 1998.

[6] K. Cheung, S. Schultz, W. Luk, "A large-scale spiking neural network accelerator for FPGA systems," ICANN, 2012.

[7] S. Han et al., "Hardware acceleration of LSTM neural networks and their applications," ACM TECS, 2020.

[8] A. Sripad, D. Sylvester, D. Blaauw, "FPGA-based spiking neural network: design and challenges," DATE, 2020.